

Async Background Processing in FastAPI

BackgroundTasks vs. `asyncio.to_thread` vs. Celery — and how document indexing uses them. (LearnMate RAG)

1. The problem

The `/upload` route currently does `await index_document(...)` inline. `index_document` = chunk → embed every chunk → write to Mongo + Qdrant → flip status to `ready`. For a large PDF that's **several seconds**, and the HTTP response does not return until it finishes. The user's browser just spins.

The idea: respond immediately with `{document_id, status: "processing"}`, do the indexing *after* the response is sent, and let the frontend **poll** a status endpoint until it flips to `ready`.

2. How FastAPI BackgroundTasks works

FastAPI injects a special parameter. You register a function on it, and FastAPI runs that function **after the response has been sent**:

```
from fastapi import BackgroundTasks

@router.post("/upload")
async def upload(background_tasks: BackgroundTasks, file: UploadFile = File(...), ...):
    doc = await save_document_metadata(...) # status = "processing"
    text = (await process_uploaded_file(file))["text"] # read file NOW, in the request
    background_tasks.add_task(index_document, doc.id, email, text, session_id) # scheduled, NOT awaited
    return {"document_id": doc.id, "status": "processing"} # returns NOW
```

- `add_task(fn, *args)` **registers** the call; it does not run yet.
- FastAPI sends the response first, **then** runs the task.
- Exceptions in the task **cannot** reach the client (the response is gone) — the task must record its own failure. `index_document` already does this: its `try/except` sets status to `failed`.

3. The key question: what does "background" actually use?

BackgroundTasks does NOT spawn a new thread for an async task. "Background" means **later in time** (after the response is sent), **not** "on a separate thread." An `async def` task runs on the **same single event loop**, in the same thread — just deferred until the response has been flushed.

The one nuance — it depends on whether the task is async or sync. This is what Starlette (FastAPI's foundation) does internally:

```
# roughly what Starlette does for each background task
if iscoroutinefunction(func):
    await func(*args) # async -> runs ON THE EVENT LOOP, deferred (no new thread)
else:
    await run_in_threadpool(func) # sync -> runs in a THREADPOOL, deferred
```

- **async def task** → awaited on the event loop after the response. **No new thread.**
- **plain def task** → pushed to a threadpool so a blocking sync function doesn't freeze the loop. **This one does use a thread** — because it has to.

4. The three tools compared

Tool	Question it answers	Mechanism	User waits?	Survives restart?
<code>asyncio.to_thread(fn)</code>	WHERE does it run?	A separate worker thread , right now (you <code>await</code> the result)	Yes — the request still blocks on it	n/a (in-request)
<code>BackgroundTasks.add_task(fn)</code>	WHEN does it run?	Same event loop (async) or threadpool (sync), after the response	No — response returns first	No — same process; lost if server dies
Celery / ARQ / RQ	WHERE (a different one)	A separate worker process via a broker (Redis)	No	Yes — durable queue, retries, scaling

One-liners:

- **to_thread** = "don't freeze the server while I finish this — but I'll still make you wait for the answer." (a *place*: another thread)
- **BackgroundTasks** = "here's your answer now; I'll do the slow part after you've left." (a *time*: deferred)
- **Celery** = "here's your answer now; a separate, durable worker does the slow part — and retries if it crashes." (a *place*, a separate process)

5. How they compose (this is the important part)

`index_document` is an `async def`. As a background task it runs **on the event loop** after the response — but inside it, the CPU-bound embedding is offloaded with `to_thread`. The two concepts solve different problems and stack cleanly:

```

1. response sent to user                                     ← instant (BackgroundTasks: the WHEN)
2. THEN index_document runs on the event loop:
    await asyncio.to_thread(generate_embeddings_batch, chunks) ← CPU work -> worker thread (to_thread: the
    await store_embeddings_batch(...)                          ← async Qdrant I/O -> loop
    await documents_collection.insert_many(...)                ← async Mongo I/O -> loop
    set status = "ready" (or "failed")

```

Result: the user gets an **instant response**, *and* the heavy embedding still runs on a **separate thread** so it doesn't freeze the loop for other users.

6. The gotcha

Because an async background task runs **on the event loop**, if it did blocking work **without** offloading — e.g. `model.encode(chunks)` directly with no `to_thread` — it would **freeze the event loop for every other user**, just *after* the response instead of before. The name "BackgroundTasks" tricks people into thinking it's on another thread, so blocking is fine. It isn't. Our `index_document` is safe **only because** it already offloads its CPU step.

7. Mental model — hold all three in your head

Tool	Axis	Plain meaning
<code>to_thread</code>	a place	run this blocking thing on another thread so the loop stays free
<code>BackgroundTasks</code>	a time	run this after I've answered, on the same loop (or a threadpool if sync)
Celery	a place (separate)	run this in a different process entirely, durably

BackgroundTasks is the lightest: no new thread, no new process — just deferred execution on the loop you already have. That's why it's perfect for "kick off indexing and reply," and why it still needs `to_thread` inside for the CPU part.